

实验报告

1. 此前是否有多人协同开发的经历？如果有，是使用什么方式分工协作的？

有。三年前与人协同开发了开源项目 [Generals.OI](#)。项目除最初版本外，均使用 git 协作开发。

2. Git 为什么要设计“暂存-提交”两个步骤？

在“修改文件”和“提交形成 commit”之间，git 加入了暂存区。这样设计的主要原因有两点：**第一**，方便在提交前检查内容。暂存区使得开发者可以使用 `git diff --staged` 或者 VSCode Git 插件中的对应功能来检查即将提交的修改，避免误提交测试文件、无关修改等。**第二**，可以选择性地提交修改的内容。开发过程中工作区内可能存在多种修改，如修复bug、更改文档、测试等，如果不经过暂存就直接提交，会导致混入部分不必提交的更改。而且，一次 git 提交应该只完成一件比较完整的事情（比如实现单个功能），这样能使提交历史清晰，而暂存的步骤方便开发者把不同目的的更改拆分成多个提交。

3. `git branch` 与 `git branch -a` 的区别是什么？

```
List or delete (if used with -d) the remote-tracking branches. Combine with --list to match the optional pattern.

-a, --all
    List both remote-tracking branches and local branches. Combine with --list to match optional pattern(s).

git branch
  feature
  * main
alvar_santagar@LAPTOP-MDPF0LBV:~/projects/ics-course-gitlab$ git branch -a
  feature
  * main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
alvar_santagar@LAPTOP-MDPF0LBV:~/projects/ics-course-gitlab$ git branch --help
```

如图，执行 `git branch` 只显示本地分支，其中带*号的为当前分支；执行 `git branch -a` 则显示本地分支和远程跟踪分支。此处显示的远程跟踪分支不是通过直接访问服务器得到的，而是本地的记录，需要通过 `git fetch` 来同步。

4. 《Commit message 和 Change log 编写指南》内容概括

这篇文章主要介绍如何编写规范的提交说明，以改善 git 记录，并利用提交记录生成版本更新日志。其中采用的提交格式规范为 Angular 规范，基本格式为：

```
<type>(<scope>): <subject>

<body>

<footer>
```

其中：

- `type` 表示提交类型，常见的有 `feat`、`fix`、`docs`、`style`、`refactor`、`test`、`chore` 等；
- `scope` 表示修改影响的范围，可以省略；
- `subject` 是对本次修改的简短说明，注意以动词开头，使用第一人称现在时，首字母小写，结尾不加句号；
- `body` 进一步解释修改原因和具体内容，说明代码变动的动机，以及与以前行为的对比；
- `footer` 只用于两种情况：说明不兼容性变更或与本次提交相关联的 issue。

另有一种特殊情况，即撤回以前提交，此时必须以 `revert` 开头，且 `body` 格式也固定。文章还介绍了 Commitizen、提交信息检查工具 (validate-commit-msg) 和利用符合 Angular 规范的 commit 自动生成 Change log 的工具 ([conventional-changelog](#))。

5. 《Gitflow 使用规范》内容概括

这篇文章介绍了一种以 git 为基础，通过不同分支管理开发、测试、发布和线上修复的工作方式。Gitflow 中主要包括以下分支：

分支	主要作用
master	保存正式发布的稳定代码，分支上带有 tag
develop	开发过程中的操作分支
feature	开发某个具体的新功能
release	进行发布前测试、版本调整和问题修复
hotfix	紧急修复正式环境中的问题
bugfix	修复测试阶段发现的问题

`feature` 分支从 `develop` 创建，开发完成后合并回 `develop`，然后被删除。此处合并使用 `--no-ff`，保留明确的合并提交，使提交历史能够看出该功能是在独立分支中开发的。开发达到测试阶段时，将新代码从 `develop` 推送至 `release`，进行测试验证。对于测试中的缺陷，创建单独的 `bugfix` 分支进行修复，完成后合并回 `release` 并删除 `bugfix`。测试完成上线时，更新 `master` 分支代码后，`release` 合并 `master` 代码，`master` 再合并 `release`。发布后确认代码缺陷时，基于最新 `master` 分支创建 `hotfix` 分支进行修复，完成后合并回 `master` 并删除 `hotfix`。

6. 为什么要学习 git?

我认为 git 最重要的价值在于帮助开发者有组织地管理变化。

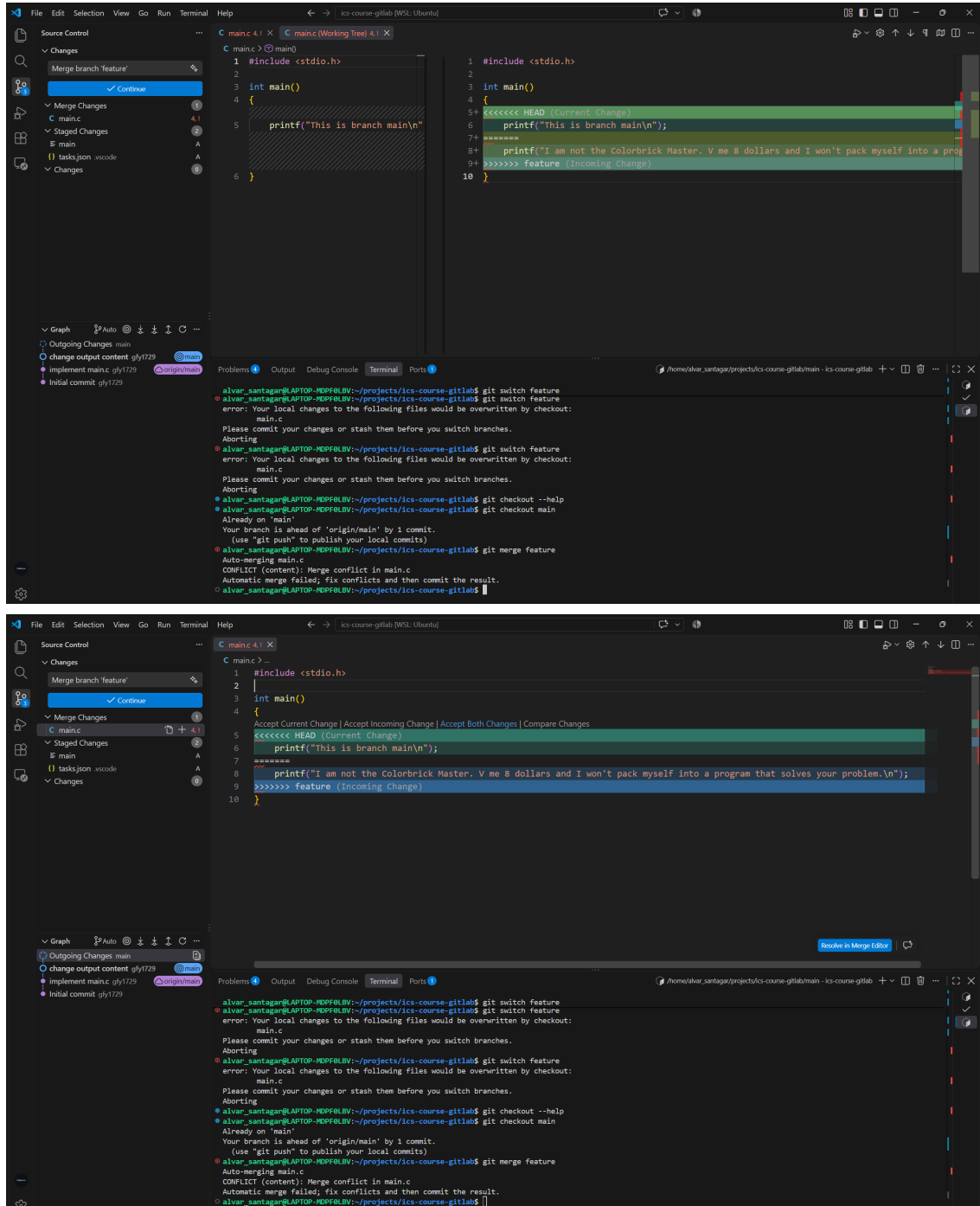
首先，git 能够完整记录代码的发展过程。每次提交都可以看作一个版本节点。当程序出现问题时，可以比较不同版本、查找修改来源，必要时还可以恢复到以前的状态。这远比手工保存“最终版”“最终版2”等可靠。

其次，git 可以降低尝试新方案的风险。开发者可以创建一个新分支，在不影响稳定代码的情况下进行实验。如果实验成功，就将其合并；如果失败，则可以直接放弃该分支。

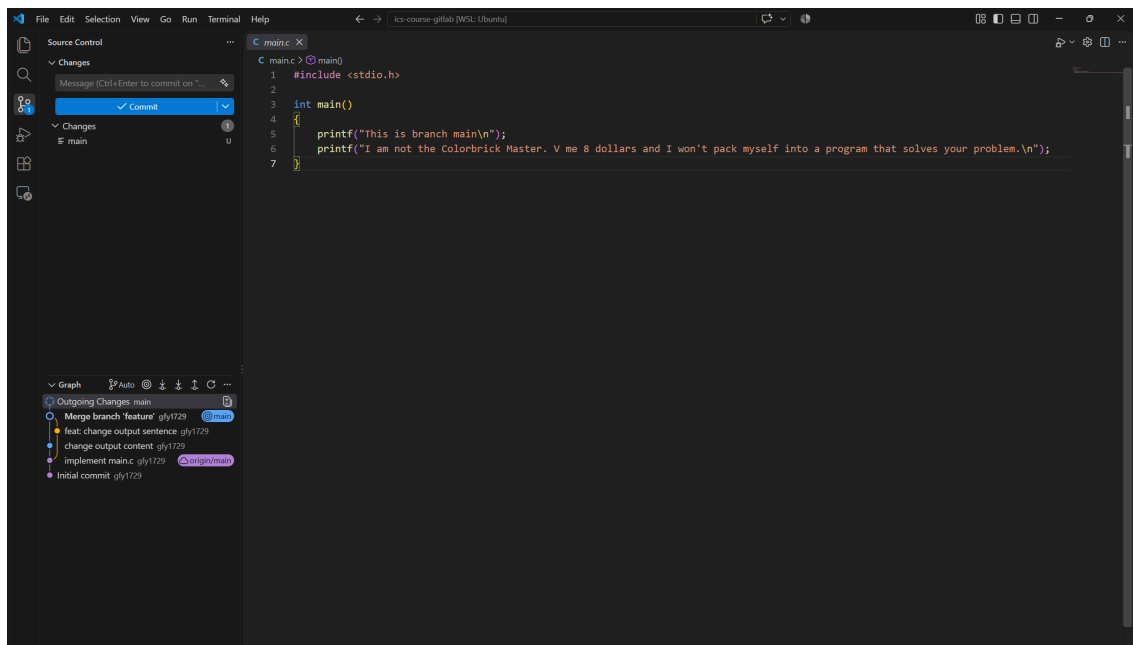
再次，git 是团队协作的重要基础。多人可以分别开发不同功能，整合工作成果。规范的 Commit message 能够说明修改了什么、为什么修改，Gitflow 则进一步规定“修改应该在哪个分支中进行以及最终流向哪里”。

此外，git 还连接着代码审查、持续集成、自动测试和软件发布等现代开发流程。清晰的提交历史和版本标签不仅方便开发，也为测试、部署、问题追踪和版本回滚提供依据。

7. 实验记录（新建分支、合并分支、解决冲突）



如上图，由于在 `main` 和 `feature` 两个分支中对 `main.c` 的同一行（`printf` 语句）做了不同的更改，在将 `feature` 合并至 `main` 时产生了冲突。



通过手动更改，同时保留 `main.c` 两个版本的不同 `printf` 语句，解决了冲突，成功完成 `merge` 操作，并推送至 GitHub。